

SCINTILLA

Literary & Arts Magazine

Backend & Integration Layer

Product Requirements Document

Version 1.0 | Production-Ready

Classification: Internal Engineering

Senior Backend Architect Review

SECTION 01 EXECUTIVE OVERVIEW

1. Executive Overview

1.1 Project Purpose

Scintilla is a student-run literary and arts magazine that collects creative submissions through a two-page web interface. The frontend — `index.html` (landing) and `submit.html` (submission form) — is already fully built and must not be modified. This PRD defines the complete backend and integration layer that will power the submission pipeline, file storage, and data tracking system.

1.2 Scope Statement

This document covers backend architecture ONLY. No frontend changes are included or implied. The backend must seamlessly attach to the existing HTML forms without requiring any modifications to them.

1.3 System Goals

- Accept and validate multi-file, multi-category form submissions from `submit.html`
 - Upload submitted files to a designated Google Drive account, organised into category subfolders

Log all submission metadata into a Google Sheet for editorial review

Provide secure, validated API endpoints with proper error handling

Support concurrent submissions without data loss or race conditions

Be deployable to a cloud host with HTTPS, environment-variable secrets, and structured logging

1.4 Out of Scope

- Any modification to `index.html` or `submit.html`
 - A user authentication/login system (submissions are anonymous or lightly identified)

A CMS, editorial review UI, or admin dashboard (managed directly in Google Sheets)

Email confirmation to submitters (optional future enhancement, not in v1)

SECTION 02 SYSTEM ARCHITECTURE

2. System Architecture

2.1 Architecture Overview

The system follows a lightweight, serverless-compatible three-tier architecture: a static frontend tier, a Node.js REST API tier, and a Google Workspace backend tier (Drive + Sheets). All tiers communicate over HTTPS with JWT-signed service credentials.

2.2 Technology Stack

Layer / Concern	Technology Choice
Runtime	Node.js v20 LTS
Framework	Express.js 4.x
File Upload	Multer (memory storage, 25 MB limit)
Google APIs	googleapis npm package (v6+)
Authentication	Google Service Account + JWT (googleapis)
Validation	Joi schema validation
Logging	Winston + daily-rotate-file
Environment	dotenv for local, platform secrets for prod
Hosting	Render (primary) or Vercel (serverless fallback)
HTTPS	Enforced via platform + Helmet.js middleware
Rate Limiting	express-rate-limit (100 req / 15 min per IP)
CORS	cors package, whitelist of allowed origins

2.3 Folder Structure

The following is the canonical backend project layout. Every directory has a single responsibility.

```
scintilla-backend/
├─ src/
│   ├── routes/
│   │   ├── submit.js           # POST /submit handler
│   │   └─ status.js           # GET /status health check
│   ├── controllers/
│   │   └─ submissionController.js
│   ├── services/
│   │   ├── driveService.js     # Google Drive upload logic
│   │   └─ sheetsService.js     # Google Sheets append logic
│   ├── middleware/
│   │   ├── validate.js         # Joi schema validation
│   │   ├── rateLimiter.js      # Per-IP rate limiting
│   │   └─ security.js          # Helmet + CORS + HTTPS redirect
│   ├── utils/
│   │   ├── logger.js           # Winston logger singleton
│   │   └─ fileRenamer.js       # Name_Category_Title formatting
```

```
| | └─ retry.js           # Exponential back-off helper
| | └─ config/
| |   └─ index.js        # Centralised env var access
| |   └─ app.js          # Express app factory
| └─ .env.example        # Template (no real secrets)
| └─ .gitignore          # Excludes .env, node_modules
| └─ package.json
| └─ server.js           # Entry point
```

2.4 Component Interaction Diagram

At a high level, each HTTP request travels through the following layers before touching any external service:

```
[submit.html Form]
|
|  multipart/form-data  POST /submit
v
[Security Middleware] - HTTPS redirect, CORS, Helmet, Rate-limiter
|
v
[Validation Middleware] - Joi schema, file-type check, size check
|
v
[SubmissionController]
|           |
v           v
[DriveService] [SheetsService]
|           |
v           v
[Google Drive] [Google Sheets]
|           |
|_____|_____|
|
v
[JSON Response → submit.html]
```

SECTION 03 API DESIGN

3. API Design

3.1 Base URL

All API routes are prefixed with /api/v1. In production this resolves to <https://api.scintilla.yourdomain.com/api/v1>.

3.2 Endpoints Summary

Method	Path	Purpose
POST	/api/v1/submit	Submit one or more creative works
GET	/api/v1/status	Health check — returns server + Google API status
GET	/api/v1/version	Returns app version and deployment date

3.3 POST /api/v1/submit

Request Format

Content-Type: multipart/form-data

Field	Description / Validation
name	string — full name of submitter (required)
email	string — valid email address (required)
course	string — degree programme / course (required)
year	string — year of study, e.g. "2" or "Postgrad" (required)
submissions	JSON string — array of submission objects (required)
files[]	binary — one file per submission entry (required)

Each object in the submissions JSON array must contain:

Key	Value
category	string — one of: poetry, articles, artwork, photography, fiction, nonfiction
title	string — work title, max 120 chars
description	string — brief synopsis, max 500 chars

Successful Response (HTTP 200)

```
{
  "status": "success",
  "message": "Submission received.",
  "submissionId": "sub_1718200000_abc123",
}
```

```
"entries": [  
  {  
    "title": "Ode to Chennai",  
    "category": "poetry",  
    "driveUrl": "https://drive.google.com/file/d/...",  
    "sheetRow": 42  
  }  
]}
```

Error Responses

HTTP Status	Meaning
400 Bad Request	Missing required field or invalid JSON in submissions array
413 Payload Too Large	File exceeds 25 MB per file or 75 MB total request
415 Unsupported Media	File MIME type not in allowed list
429 Too Many Requests	Rate limit exceeded (100 req / 15 min per IP)
500 Internal Server	Google API failure or unexpected server error
503 Service Unavail	Google Drive or Sheets quota exceeded

All error responses share this body shape:

```
{  
  "status": "error",  
  "code": "VALIDATION_FAILED",  
  "message": "Human-readable description",  
  "field": "email"    // present for validation errors only  
}
```

3.4 GET /api/v1/status

Returns a JSON object with server uptime, the version string, and connectivity checks for Google Drive and Google Sheets. This endpoint requires a static Bearer token stored in env vars and is intended for internal monitoring only — it must not be publicly advertised.

```
{  
  "status": "ok",  
  "uptime": 38492,  
  "version": "1.0.0",  
  "services": {  
    "googleDrive": "connected",  
    "googleSheets": "connected"  
  }  
}
```

SECTION 04 DATA FLOW

4. Data Flow

4.1 Step-by-Step Submission Flow

STEP 1 – User completes submit.html form
↓ Browser collects all fields + files via JS cart logic

STEP 2 – Browser sends POST /api/v1/submit
↓ multipart/form-data (text fields + binary files)
↓ Content-Length header validated client-side before send

STEP 3 – Security Middleware intercepts request
↓ HTTPS enforced (redirect HTTP → HTTPS)
↓ Origin validated against CORS whitelist
↓ Helmet headers applied (CSP, X-Frame-Options, etc.)
↓ Rate limiter checks IP counter in memory

STEP 4 – Multer parses the multipart body
↓ Files held in memory (Buffer) – never written to disk
↓ Per-file size check (25 MB limit)
↓ MIME-type sniffing via file-type package

STEP 5 – Joi validation runs on all text fields
↓ submissions JSON parsed and array-validated
↓ Rejects unknown categories immediately (400)
↓ Checks file count matches submissions array length

STEP 6 – SubmissionController creates a unique submission ID
sub_{unixTimestamp}_{randomHex6}
↓ Iterates submissions array in order

STEP 7 – For each entry → DriveService
↓ Looks up or creates category subfolder in Drive
↓ Renames file: {Name}_{Category}_{Title}.{ext}
↓ Streams Buffer to Drive via googleapis resumable upload
↓ Sets file permissions to 'reader for anyone with link'
↓ Returns shareable Drive URL

STEP 8 – For each entry → SheetsService
↓ Appends one row to Google Sheet
↓ Row: [submissionId, timestamp, name, email, course, year, category, title, description, driveUrl]

STEP 9 – Both services return results

↓ Controller assembles response object
↓ Winston logs submission at INFO level

STEP 10 – HTTP 200 JSON response returned to browser
↓ submit.html displays success state to user

4.2 File Renaming Logic

Files are renamed before upload to ensure human-readable, conflict-resistant filenames in Drive. The logic lives in `src/utils/fileRenamer.js`.

```
// fileRenamer.js
function buildFileName(name, category, title, originalName) {
  const ext = path.extname(originalName).toLowerCase();
  const sanitize = s => s.trim()
    .replace(/^[a-zA-Z0-9\s]/g, "")
    .replace(/\s+/g, "_")
    .substring(0, 40);
  return `${sanitize(name)}_${sanitize(category)}_${sanitize(title)}${ext}`;
  // e.g. "Priya_Sharma_Poetry_Ode_to_Chennai.pdf"
}
```

4.3 Google Drive Folder Structure

The backend creates and maintains the following folder hierarchy under the configured root folder ID:

```
📁 Scintilla Submissions (root – ID stored in env var)
├── 📁 Poetry
├── 📁 Articles
├── 📁 Artwork
├── 📁 Photography
├── 📁 Fiction
└── 📁 Non-Fiction
```

On startup, `DriveService.ensureFolders()` verifies all category folders exist and caches their IDs in a module-level Map. This prevents repeated Drive API calls during the request lifecycle.

4.4 Google Sheets Row Format

Column	Field	Example Value
A	submissionId	sub_1718200000_abc123
B	timestamp	2024-06-12T14:30:00.000Z

C	name	Priya Sharma
D	email	priya@college.edu
E	course	BA English Literature
F	year	3
G	category	Poetry
H	title	Ode to Chennai
I	description	A meditation on...
J	driveUrl	https://drive.google.com/file/d/...

SECTION 05 SECURITY IMPLEMENTATION

5. Security Implementation

⚠️ NON-NEGOTIABLE: All items in this section are mandatory. No exceptions.
Deviation from any requirement below constitutes a production security violation.

5.1 Secret Management

Zero secrets in source code. The following environment variables are required and must NEVER appear in any committed file:

Environment Variable	Purpose
GOOGLE_SERVICE_ACCOUNT_JSON	Full service account JSON (stringified)
GOOGLE_DRIVE_ROOT_FOLDER_ID	Drive folder ID for submissions
GOOGLE_SHEETS_SPREADSHEET_ID	Target spreadsheet ID
STATUS_BEARER_TOKEN	Static token protecting /status endpoint
ALLOWED_ORIGINS	Comma-separated CORS whitelist
NODE_ENV	"production" or "development"
PORT	Server port (default 3000)

A .env.example file lists all required keys with placeholder values. The .gitignore must contain .env, *.pem, and any credential file patterns. CI/CD pipelines must inject secrets via platform secret stores (Render Environment Variables, GitHub Secrets, etc.) — never via .env files checked in.

5.2 HTTPS Enforcement

```
// security.js middleware
if (process.env.NODE_ENV === "production") {
  app.use((req, res, next) => {
    if (req.headers["x-forwarded-proto"] !== "https") {
      return res.redirect(301, `https://${req.headers.host}${req.url}`);
    }
    next();
  });
}
```

5.3 HTTP Security Headers (Helmet.js)

Header	Policy
Content-Security-Policy	Restricts resource origins; blocks inline scripts from injected content
X-Frame-Options	DENY — prevents clickjacking in iframes
X-Content-Type-Options	nosniff — prevents MIME-type confusion attacks
Referrer-Policy	strict-origin-when-cross-origin
Strict-Transport-Security	max-age=31536000; includeSubDomains
X-Powered-By	Removed — hides Express fingerprint

5.4 CORS Configuration

```
// cors middleware
const corsOptions = {
  origin: (origin, cb) => {
    const allowed = process.env.ALLOWED_ORIGINS.split(",");
    if (!origin || allowed.includes(origin)) cb(null, true);
    else cb(new Error("Not allowed by CORS"));
  },
  methods: ["GET", "POST"],
  allowedHeaders: ["Content-Type", "Authorization"],
  maxAge: 600
};
```

5.5 Rate Limiting

Applied globally at the Express app level before any route handlers.

```
const rateLimit = require("express-rate-limit");
const limiter = rateLimit({
```

```
windowMs: 15 * 60 * 1000, // 15 minutes
max: 100,
standardHeaders: true,
legacyHeaders: false,
handler: (req, res) => {
  logger.warn("Rate limit exceeded", { ip: req.ip });
  res.status(429).json({ status: "error", code: "RATE_LIMITED" });
}
});
```

5.6 File Upload Security

- MIME-type sniffing with the file-type package (reads magic bytes, not just extension)
 - Allowlist of accepted MIME types: PDF, PNG, JPEG, GIF, WEBP, TIFF, MP3, MP4, DOCX

25 MB per-file hard limit; 75 MB total request body limit

Files buffered in memory only — no temporary disk writes eliminates path-traversal risk

Filenames are fully sanitised before use in Drive API calls (see fileRenamer.js)

No file content is ever returned to the client

5.7 Injection & IDOR Prevention

- All text fields pass through Joi schema validation before use — no raw user input ever reaches Google API calls
 - The submissions array is validated element-by-element; any malformed entry rejects the entire request

The backend does not expose any resource IDs from Drive or Sheets in a way that allows enumeration

There is no ownership model (submissions are write-only); however, Drive file permissions are set to 'restricted' (magazine team only) by default — the shareable link is only in the Sheet

No SQL or NoSQL database is used; Sheets API calls are append-only, eliminating injection surfaces

5.8 Secrets Scanning in CI/CD

- Add detect-secrets or gitleaks as a pre-commit hook and CI step
 - Fail the pipeline if any string matching Google API key patterns is found in committed files

Rotate all credentials immediately if a scan detects an existing leak

SECTION 06 LOGGING & MONITORING

6. Logging & Monitoring

6.1 Winston Logger Configuration

Winston is configured with two transports: Console (development) and DailyRotateFile (production). Log files rotate daily and are retained for 14 days.

```
// src/utils/logger.js
const logger = winston.createLogger({
  level: process.env.NODE_ENV === "production" ? "info" : "debug",
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new DailyRotateFile({
      filename: "logs/app-%DATE%.log",
      datePattern: "YYYY-MM-DD",
      maxFiles: "14d"
    })
  ]
});
```

6.2 Structured Log Events

Event Code	Level	Data Logged
SUBMISSION_RECEIVED	INFO	IP, timestamp, number of files, total size
SUBMISSION_SUCCESS	INFO	submissionId, Drive URLs, sheet row numbers
VALIDATION_FAILED	WARN	IP, failed field, rejection reason
DRIVE_UPLOAD_FAILED	ERROR	submissionId, filename, error message, retry count
SHEETS_APPEND_FAILED	ERROR	submissionId, row data, error message
RATE_LIMIT_EXCEEDED	WARN	IP address, endpoint, timestamp
AUTH_ERROR	ERROR	Endpoint, header present (bool), timestamp
LARGE_PAYLOAD	WARN	IP, total bytes, file count
STARTUP	INFO	Version, Drive folder IDs verified, Sheet ID

6.3 Suspicious Activity Detection

The following patterns trigger WARN or ERROR log events and, in production, should be routed to an alerting channel (e.g. Slack webhook or email):

- More than 10 submissions from a single IP within 15 minutes (rate limiter fires, triggers WARN log)
 - Repeated MIME-type validation failures from the same IP — suggests probing or fuzzing

File upload with size exactly at or near the 25 MB limit from an unknown origin
CORS rejection — origin not on whitelist means an external site is attempting to post forms
Any 5xx error spike — alert if error rate exceeds 5 errors/minute over any 5-minute window

6.4 Health Check Monitoring

The GET /api/v1/status endpoint should be polled every 60 seconds by an uptime monitor (e.g. UptimeRobot, Better Uptime). A failed check should send an immediate alert to the magazine admin email.

SECTION 07 FAILURE HANDLING & RETRY LOGIC

7. Failure Handling & Retry Logic

7.1 Retry Strategy

Google API calls (Drive upload, Sheets append) are wrapped in an exponential back-off retry helper. Transient failures (5xx from Google, network timeouts) are retried up to 3 times before the request is considered failed.

```
// src/utils/retry.js
async function withRetry(fn, maxAttempts = 3) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await fn();
    } catch (err) {
      if (attempt === maxAttempts) throw err;
      const delay = Math.pow(2, attempt) * 500; // 1s, 2s, 4s
      logger.warn("Retrying API call", { attempt, delay, error: err.message });
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

7.2 Partial Failure Handling

If a multi-entry submission (cart with multiple works) partially succeeds — for example, two uploads succeed and one fails — the backend:

- Returns HTTP 207 Multi-Status with a per-entry status array
 - Logs the partial failure event with all successfully processed entry data

Does NOT roll back already-completed Drive uploads or Sheet rows (append-only system)

Includes the failed entry's error message in the response so the browser can display it

7.3 Google API Quota Exhaustion

The backend detects HTTP 429 or 403 quota-exceeded responses from Google APIs and returns HTTP 503 to the client with a Retry-After header of 60 seconds. The editorial team should monitor the Google Cloud Console quota dashboard for the service account project.

SECTION 08 DEPLOYMENT PLAN

8. Deployment Plan

8.1 Recommended Host: Render

Render is recommended for its zero-configuration HTTPS, managed environment variables, and persistent Node.js service support (unlike Vercel which has 10-second timeout constraints that risk large file uploads).

Setting	Value
Service Type	Web Service (not serverless)
Build Command	npm install
Start Command	node server.js
Health Check URL	/api/v1/status
Auto-Deploy	Yes, on push to main branch
Region	Singapore (closest to Indian users)

8.2 Environment Setup Steps

1. Create a Google Cloud project for Scintilla

Enable the Google Drive API and Google Sheets API

Create a Service Account; download the JSON key

Share the target Drive folder with the service account email (Editor role)

Share the Google Sheet with the service account email (Editor role)

In Render dashboard: add all required env vars from .env.example

Set GOOGLE_SERVICE_ACCOUNT_JSON to the stringified JSON key content

Set NODE_ENV=production, ALLOWED_ORIGINS=https://yourdomain.com

Deploy and confirm /api/v1/status returns {services: {googleDrive: 'connected', googleSheets: 'connected'}}

8.3 Domain & HTTPS

- Purchase domain (e.g. scintillalmag.in) and point DNS to Render service

- Render auto-provisions TLS via Let's Encrypt — no manual SSL configuration needed
- The HTTPS redirect middleware in security.js handles any HTTP traffic at the app level
- Set HSTS max-age to 31536000 (1 year) only after confirming HTTPS works correctly

8.4 Fallback: Vercel Serverless

If Render is unavailable, the codebase can be adapted for Vercel with these constraints: the Multer middleware must be replaced with Vercel's native multipart parser, the in-memory file buffer approach must be preserved (no disk writes in serverless), and the function timeout must be extended to 60 seconds in vercel.json to accommodate large uploads.

SECTION 09 FUTURE SCALABILITY

9. Future Scalability

9.1 Phase 2 Enhancements

Feature	Implementation Approach
Email Confirmations	Integrate Nodemailer or SendGrid to send submitters an acknowledgement email with their Drive link
Submission Deduplication	Hash file content (SHA-256) before upload; check Sheet for existing hash to prevent accidental duplicates
Admin Dashboard	Build a read-only React interface that queries the Google Sheet API to display submissions with filter/search
Webhook Notifications	POST to a Slack or Discord webhook on each new submission for instant editorial team alerts
File Preview	Generate PDF thumbnails for artwork submissions using sharp/pdfjs and store alongside the original
Multi-Year Support	Partition Drive folders and Sheets tabs by academic year for long-term archiving

9.2 Scaling the API

- The current architecture handles up to approximately 500 concurrent submissions on a 512 MB Render instance — sufficient for a college magazine launch cycle
 - For high-traffic periods (e.g. submission deadlines), scale the Render service to 1 GB RAM and enable horizontal scaling

If Google Sheets becomes a bottleneck (2 million cell limit), migrate logging to a PostgreSQL database (e.g. Supabase free tier) while keeping Drive as the file store

Implement a Redis-backed queue (Bull or BullMQ) if submission spikes exceed 50 concurrent requests — process uploads asynchronously and return a polling token to the client

9.3 Maintenance Guidelines

- Rotate the Google Service Account key every 90 days
 - Review Winston log files monthly; escalate any recurring ERROR patterns

Monitor Google API quota usage in Cloud Console before each submission cycle

Pin all npm dependency versions and run npm audit before each deployment

SECTION 10 IMPLEMENTATION CHECKLIST

10. Pre-Launch Implementation Checklist

Backend Core

- Express app scaffolded with all middleware in correct order
 - Multer configured with memory storage and file size limits

Joi validation schema covers all fields and the submissions array

fileRenamer.js produces sanitised, extension-preserving filenames

DriveService.ensureFolders() runs on startup and caches folder IDs

Google Integrations

- Service account credentials loaded from env var (not from disk file)
 - Drive upload uses resumable upload for files > 5 MB

Sheets append includes all 10 columns with correct column order

Retry wrapper applied to both Drive and Sheets calls

Security

- Helmet applied with all headers configured
 - CORS whitelist includes production domain only

Rate limiter active on all routes

HTTPS redirect active in production mode

.env in .gitignore; no secrets in source files

detect-secrets pre-commit hook installed

Logging

- All 9 structured log events implemented
 - Winston DailyRotateFile active in production

Log directory created on startup if it does not exist

Deployment

- All env vars set in Render dashboard
 - Health check URL returns HTTP 200

HTTPS redirect tested from an HTTP URL

Test submission reaches Drive and Sheet correctly

Rate limiter tested with rapid repeated requests